SVM-Hand On Book

SVM is capable of performing:

- Classification task
- Regression task
- Outlier detection

Unlike the logistic Regression, the SVM do not output the probabilities for each class.

It is good for complex classification, but small or medium size dataset

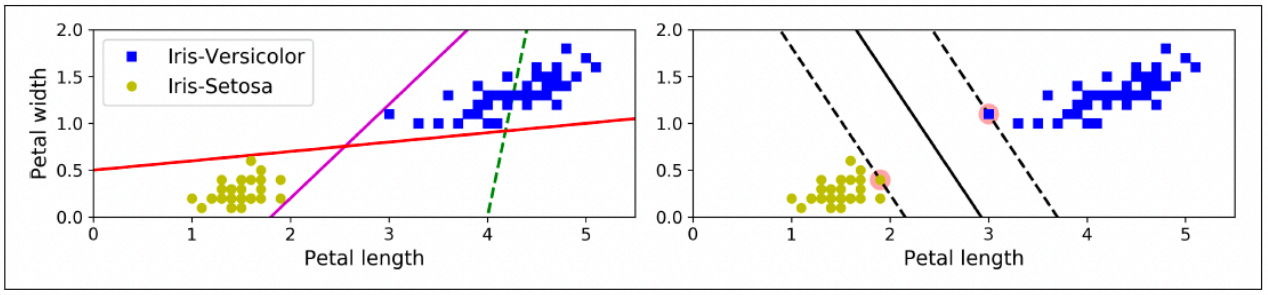
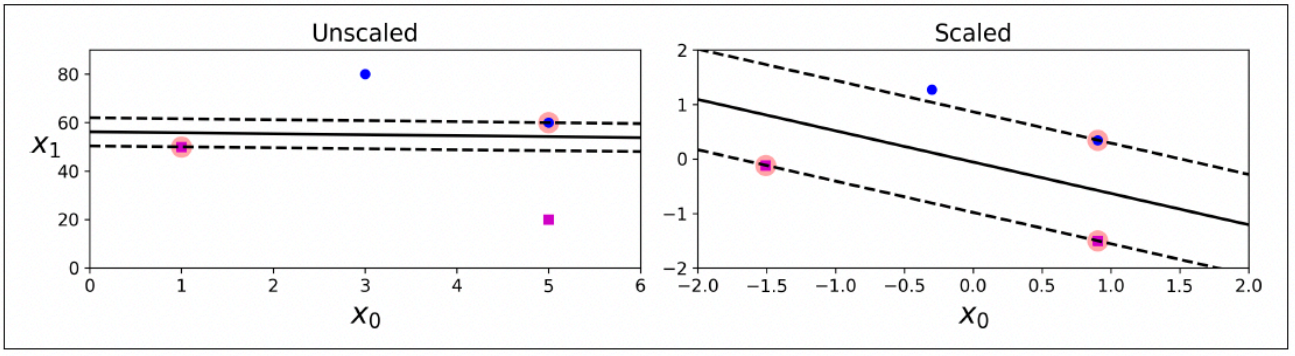


Figure 5-1. Large margin classification

Support Vectors = The instance that is circled in the figure 5-1
Moreover, SVM is sensitive to **Feature scales**



it work better with scaled instance,
we can do the scaling by using the StandardScaler in sklearn
the decision boundary looks better on the right figure.

Soft Margin Classification = Support Vector Classifier

Main issue with Hard margin Classification:

- it only works with if the data is linearly separable
- it is quite sensitive to outlier
  --> so we use the soft margin classification

```python
import numpy as np
from sklearn import datasets
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = datasets.load_iris()
X = iris["data"][:, (2, 3)]  # petal length, petal width
y = (iris["target"] == 2).astype(np.float64)  # Iris-Virginica

svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("linear_svc", LinearSVC(C=1, loss="hinge")),
    ])

svm_clf.fit(X, y)
```

Then, as usual, you can use the model to make predictions:

```python
>>> svm_clf.predict([[5.5, 1.7]])
array([1.])
```

A low C value --> wider street but more margin violation
A high C value --> smaller margin but less margin violation

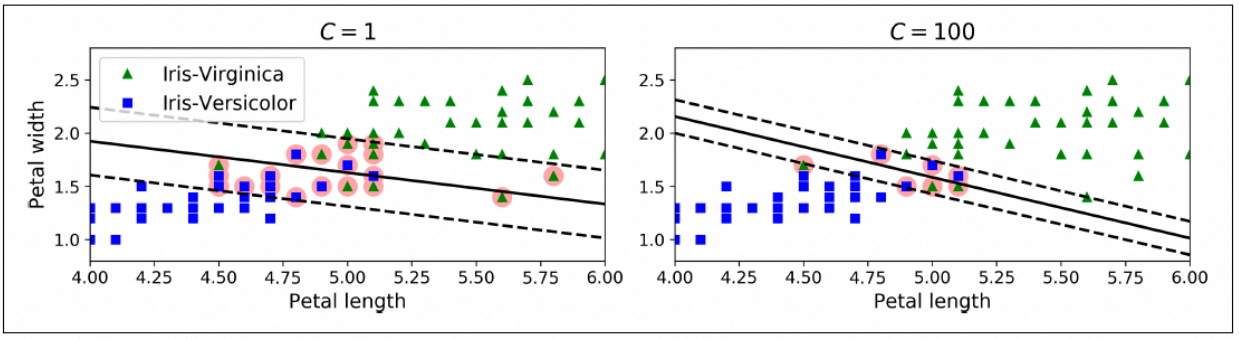


Figure 5-4. Large margin (left) versus fewer margin violations (right)

```
SVC(kernel='linear', C=1)
```

this needs more computation time for large dataset, so we should use the **SDGClassifier** instead as it comes with the **stochastic gradient descent** to train the linear **SVM** classifier.

```
SGDClassifier(loss='hinge', alpha=1/(m*C))
```

Nonlinear SVM Classifier

to handle non-linear dataset, we add more features, such as polynomial features transformer (Polynomial regression)

```
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures

polynomial_svm_clf = Pipeline([
        ("poly_features", PolynomialFeatures(degree=3)),
        ("scaler", StandardScaler()),
        ("svm_clf", LinearSVC(C=10, loss="hinge"))
    ])

polynomial_svm_clf.fit(X, y)
```

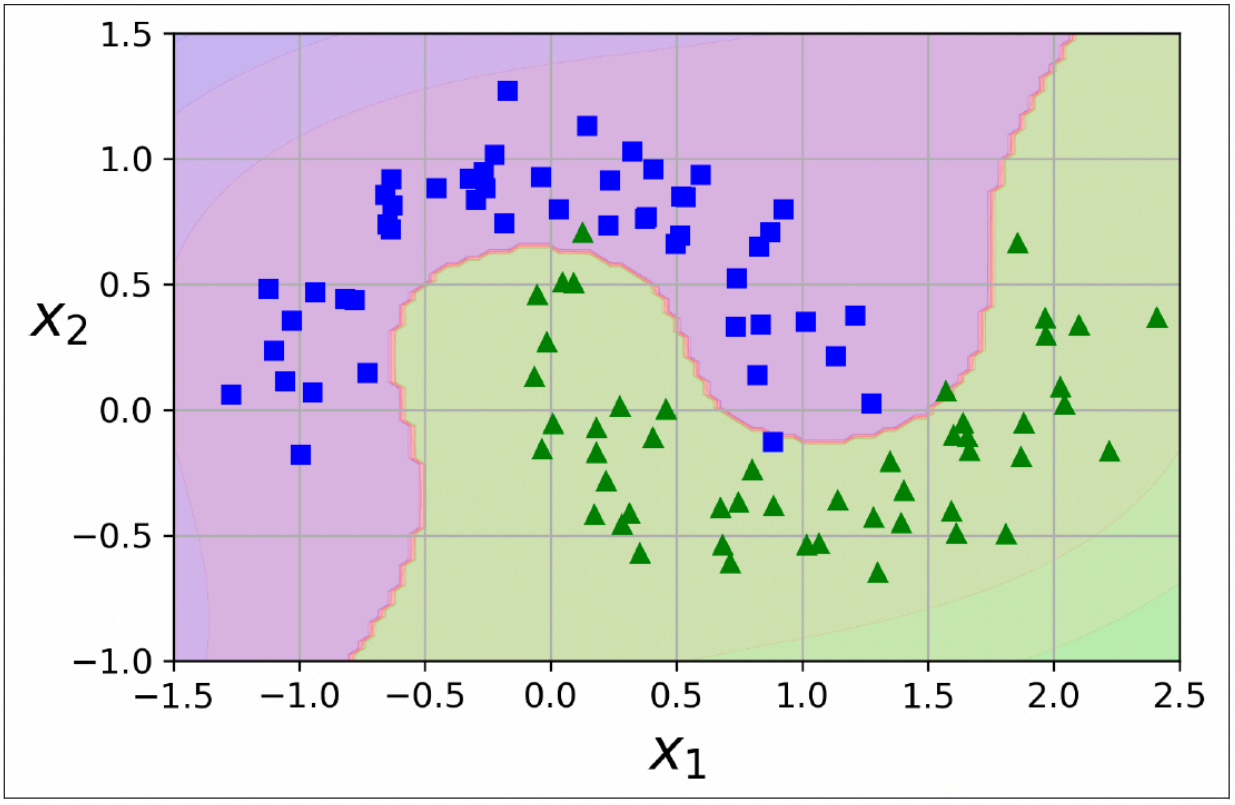


Figure 5-6. Linear SVM classifier using polynomial features

Polynomial Kernel

Polynomial Feature can be used to implement and can work great with all sort of machine learning algorithm.

But

-it cannot deal with large dataset

-with high polynomial degree, it create a lot feature which make the model too **SLOW**!

So we use **Kernel trick**

It makes it possible to get the same result as if you added many polynomial features, even with very high degree polynomials, without actually having to add them. So there is no combinatorial explosion of the number of features since you don't actually add any features.

```
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
    ])
poly_kernel_svm_clf.fit(X, y)
```

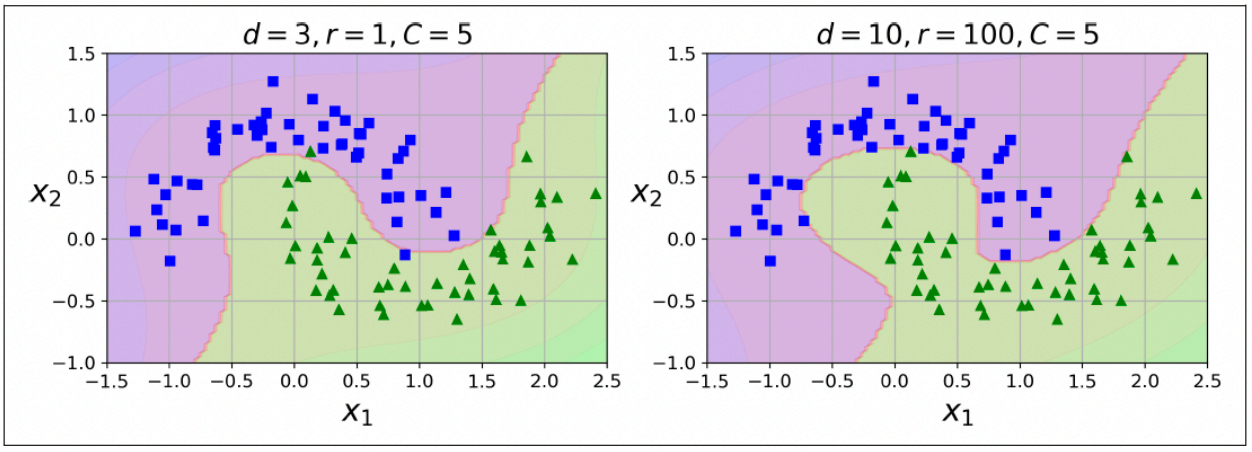


Figure 5-7. SVM classifiers with a polynomial kernel

Overfitting -> try decreasing the Degree

Underfitting -> try increasing the degree

use **GridSearch** to find the best **Hyperparameter**

Similarity Feature

it is another technique to tackle non-linear problem.

it measure how much each instance resembles a particular landmark

Equation 5-1. Gaussian RBF

$$\phi_\gamma(\mathbf{x}, \ell) = \exp\left(-\gamma \| \mathbf{x} - \ell \|^2\right)$$

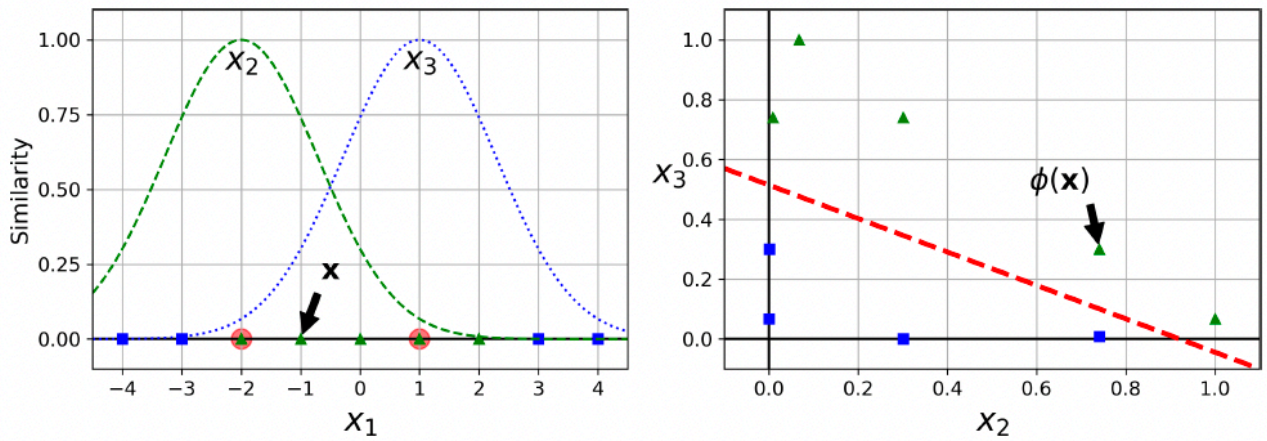


Figure 5-8. Similarity features using the Gaussian RBF

Guassian RBF Kernel

The similarity features method can be useful with any machine learning algorithm.
but it is computationally expensive to compute all the additional features, when working on the large training set.

```
rbf_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="rbf", gamma=5, C=0.001))
  ])
rbf_kernel_svm_clf.fit(X, y)
```

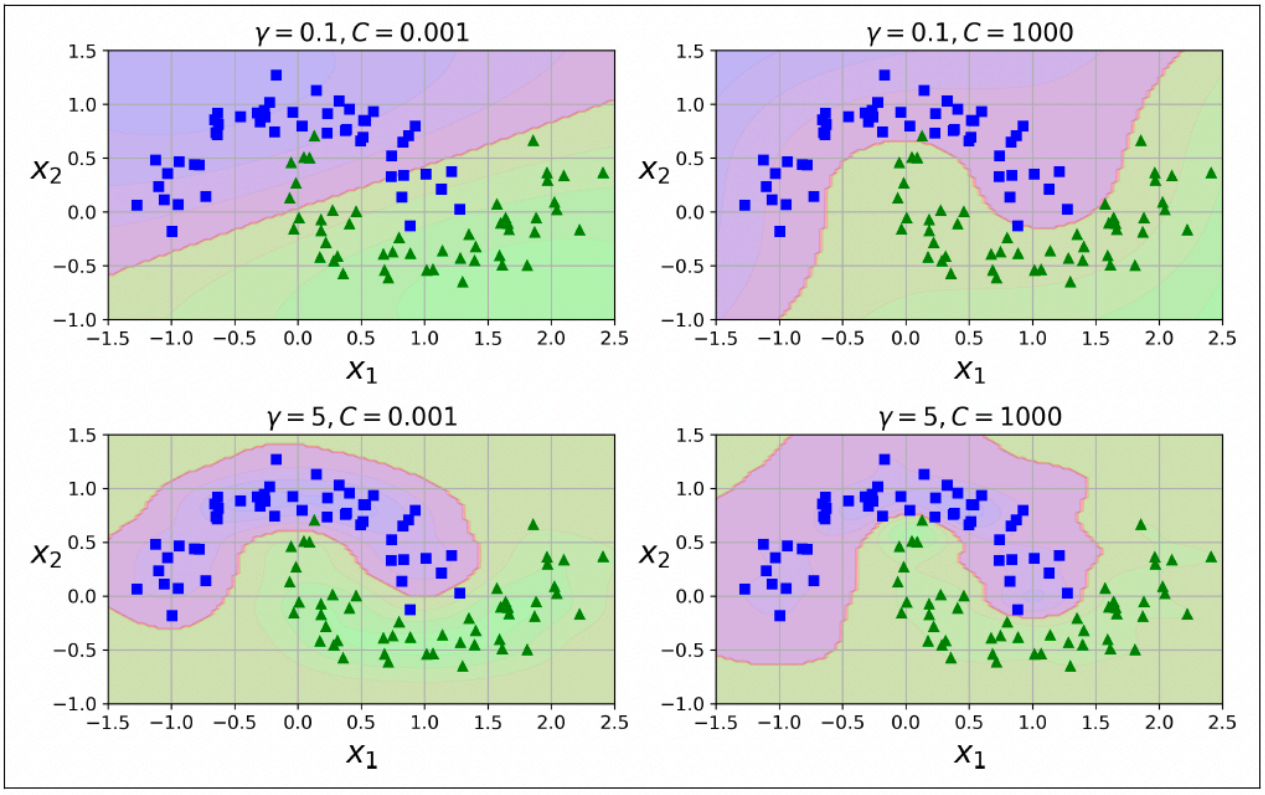


Figure 5-9. SVM classifiers using an RBF kernel

if the y in RBF
overfitting -> reduce y in RBF

Note: we learned 3 kernels (linear, rbf, poly...)
LinearSVC is faster than SVC(kernel='linear'), especially when the training set is large or if it has plenty of features.
(LinearSVC is based on **liblinear Library**)

SVM Regression

```
from sklearn.svm import LinearSVR

svm_reg = LinearSVR(epsilon=1.5)
svm_reg.fit(X, y)
```

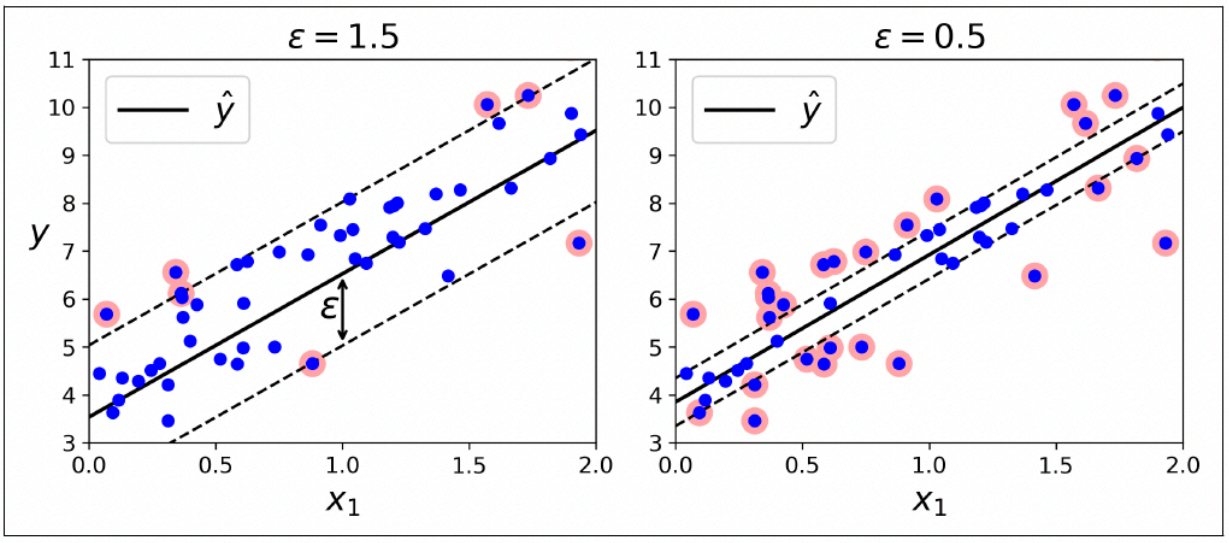


Figure 5-10. SVM Regression

```
from sklearn.svm import SVR

svm_poly_reg = SVR(kernel="poly", degree=2, C=100, epsilon=0.1)
svm_poly_reg.fit(X, y)
```

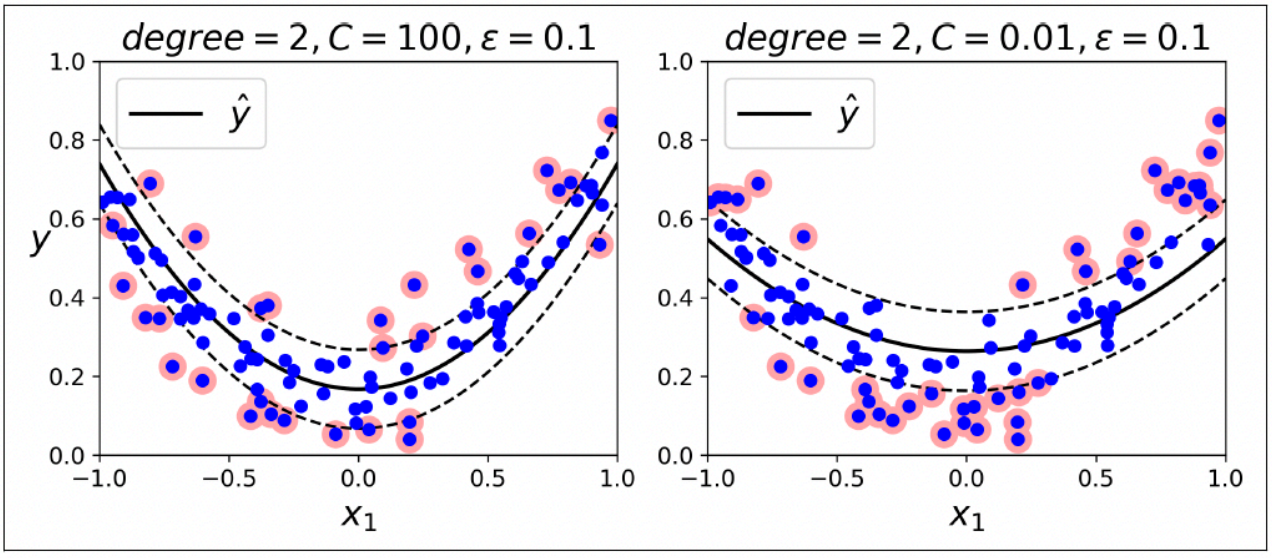


Figure 5-11. SVM regression using a 2nd-degree polynomial kernel

Hard margin linear SVM Classifier Objective

$$\underset{\mathbf{w}, b}{\text{minimize}} \quad \frac{1}{2}\mathbf{w}^T\mathbf{w}$$

$$\text{subject to} \quad t^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right) \geq 1 \quad \text{for } i = 1, 2, \cdots, m$$

Soft margin linear SVM classifier objective

$$\underset{\mathbf{w}, b, \zeta}{\text{minimize}} \quad \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{m}\zeta^{(i)}$$

$$\text{subject to} \quad t^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right) \geq 1 - \zeta^{(i)} \quad \text{and} \quad \zeta^{(i)} \geq 0 \quad \text{for } i = 1, 2, \cdots, m$$

Quadratic Programming

Equation 5-5. Quadratic Programming problem

$$\underset{\mathbf{p}}{\text{Minimize}} \quad \frac{1}{2}\mathbf{p}^T\mathbf{H}\mathbf{p} \quad + \quad \mathbf{f}^T\mathbf{p}$$

$$\text{subject to} \quad \mathbf{A}\mathbf{p} \leq \mathbf{b}$$

$$\text{where} \quad \begin{cases} \mathbf{p} & \text{is an } n_p\text{-dimensional vector } (n_p = \text{number of parameters}), \\ \mathbf{H} & \text{is an } n_p \times n_p \text{ matrix}, \\ \mathbf{f} & \text{is an } n_p\text{-dimensional vector}, \\ \mathbf{A} & \text{is an } n_c \times n_p \text{ matrix } (n_c = \text{number of constraints}), \\ \mathbf{b} & \text{is an } n_c\text{-dimensional vector}. \end{cases}$$

The Dual Problem

It is a formulated linear programming optimization model derived from the **primal problem** to give alternative solution, bound, and insight.

Equation 5-6. Dual form of the linear SVM objective

$$\underset{\alpha}{\text{minimize}} \; \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha^{(i)}\alpha^{(j)}t^{(i)}t^{(j)}\mathbf{x}^{(i)T}\mathbf{x}^{(j)} \quad - \quad \sum_{i=1}^{m}\alpha^{(i)}$$

$$\text{subject to} \quad \alpha^{(i)} \geq 0 \quad \text{for } i = 1, 2, \cdots, m$$

Equation 5-7. From the dual solution to the primal solution

$$\widehat{\mathbf{w}} = \sum_{i=1}^{m} \hat{\alpha}^{(i)} t^{(i)} \mathbf{x}^{(i)}$$

$$\hat{b} = \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^{m} \left(t^{(i)} - \widehat{\mathbf{w}}^T \mathbf{x}^{(i)} \right)$$

Note: the dual problem is faster to solve than the original primal when the number of training instances is smaller than the number of features. It make the kernel trick possible.

Kernelized SVM

Equation 5-8. Second-degree polynomial mapping

$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ \sqrt{2}\, x_1 x_2 \\ x_2^2 \end{pmatrix}$$

Equation 5-9. Kernel trick for a 2nd-degree polynomial mapping

$$\phi(\mathbf{a})^T \phi(\mathbf{b}) = \begin{pmatrix} a_1^2 \\ \sqrt{2}\, a_1 a_2 \\ a_2^2 \end{pmatrix}^T \begin{pmatrix} b_1^2 \\ \sqrt{2}\, b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2 a_1 b_1 a_2 b_2 + a_2^2 b_2^2$$

$$= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = \left(\mathbf{a}^T \mathbf{b} \right)^2$$

Equation 5-10. Common kernels

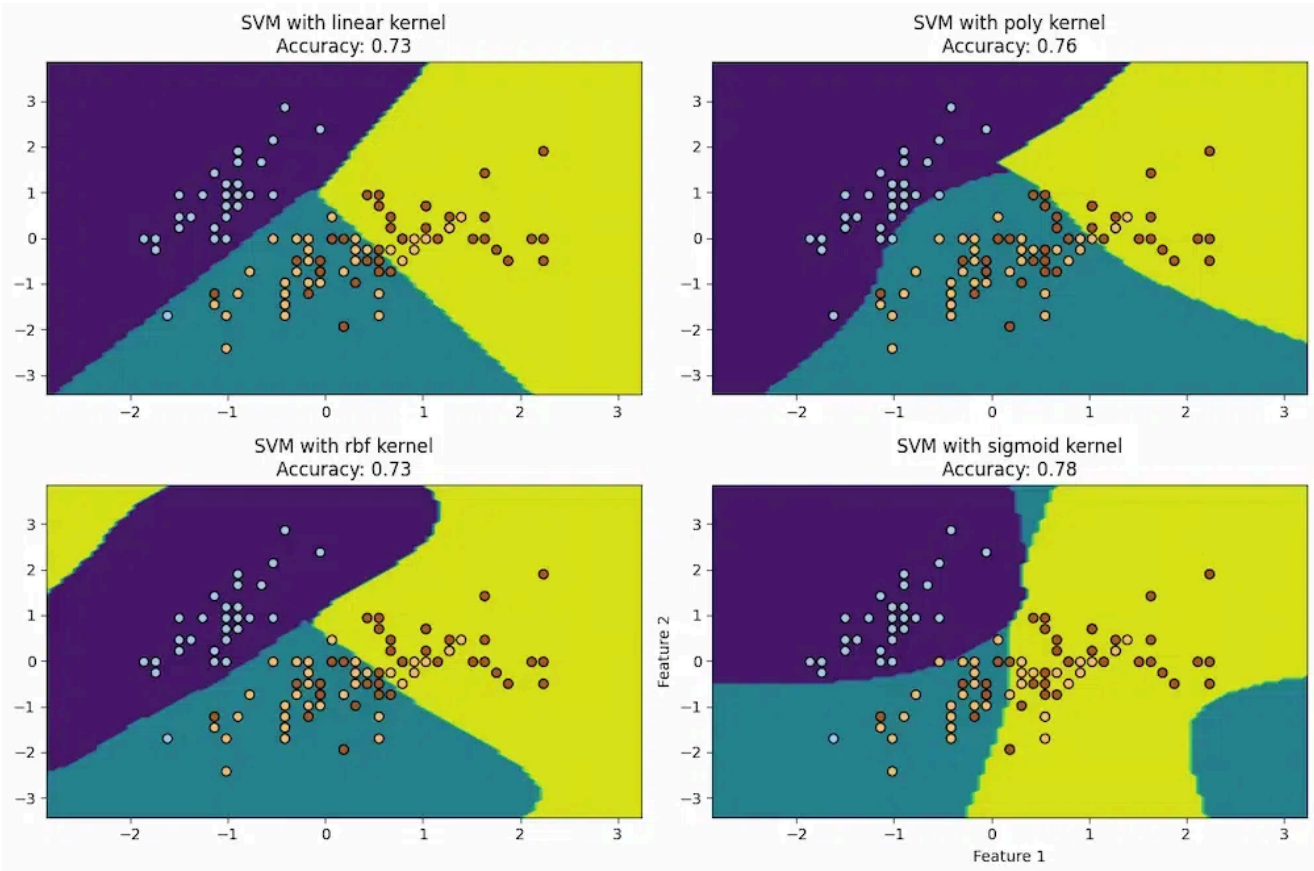
$$\text{Linear:} \quad K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^T \mathbf{b}$$

$$\text{Polynomial:} \quad K(\mathbf{a}, \mathbf{b}) = \left(\gamma \mathbf{a}^T \mathbf{b} + r\right)^d$$

$$\text{Gaussian RBF:} \quad K(\mathbf{a}, \mathbf{b}) = \exp\left(-\gamma \| \mathbf{a} - \mathbf{b} \|^2\right)$$

$$\text{Sigmoid:} \quad K(\mathbf{a}, \mathbf{b}) = \tanh\left(\gamma \mathbf{a}^T \mathbf{b} + r\right)$$

Graph for each kernel



Equation 5-11. Making predictions with a kernelized SVM

$$h_{\widehat{\mathbf{w}},\hat{b}}\left(\phi\left(\mathbf{x}^{(n)}\right)\right) = \widehat{\mathbf{w}}^T\phi\left(\mathbf{x}^{(n)}\right) + \hat{b} = \left(\sum_{i=1}^{m}\hat{\alpha}^{(i)}t^{(i)}\phi\left(\mathbf{x}^{(i)}\right)\right)^T\phi\left(\mathbf{x}^{(n)}\right) + \hat{b}$$

$$= \sum_{i=1}^{m}\hat{\alpha}^{(i)}t^{(i)}\left(\phi\left(\mathbf{x}^{(i)}\right)^T\phi\left(\mathbf{x}^{(n)}\right)\right) + \hat{b}$$

$$= \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)}>0}}^{m}\hat{\alpha}^{(i)}t^{(i)}K\left(\mathbf{x}^{(i)},\mathbf{x}^{(n)}\right) + \hat{b}$$

Equation 5-12. Computing the bias term using the kernel trick

$$\hat{b} = \frac{1}{n_s}\sum_{\substack{i=1 \\ \hat{\alpha}^{(i)}>0}}^{m}\left(t^{(i)} - \widehat{\mathbf{w}}^T\phi\left(\mathbf{x}^{(i)}\right)\right) = \frac{1}{n_s}\sum_{\substack{i=1 \\ \hat{\alpha}^{(i)}>0}}^{m}\left(t^{(i)} - \left(\sum_{j=1}^{m}\hat{\alpha}^{(j)}t^{(j)}\phi\left(\mathbf{x}^{(j)}\right)\right)^T\phi\left(\mathbf{x}^{(i)}\right)\right)$$

$$= \frac{1}{n_s}\sum_{\substack{i=1 \\ \hat{\alpha}^{(i)}>0}}^{m}\left(t^{(i)} - \sum_{\substack{j=1 \\ \hat{\alpha}^{(j)}>0}}^{m}\hat{\alpha}^{(j)}t^{(j)}K\left(\mathbf{x}^{(i)},\mathbf{x}^{(j)}\right)\right)$$

Online SVMs

Online SVM Classifier (Online learning = learning incrementally, typically as new instance arrive)
For linear SVM Classifier, one method is to use Gradient Descent to minimize the cost function (equation 5-13), which is derived from the primal problem.

Equation 5-13. Linear SVM classifier cost function

$$J(\mathbf{w}, b) = \frac{1}{2}\mathbf{w}^T\mathbf{w} \quad + \quad C\sum_{i=1}^{m} max\left(0, 1 - t^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right)\right)$$

Hinge Loss

Hinge Loss is a crucial loss function in machine learning, primarily used for training maximum-margin classifiers like **SVMs**
it **penalize** the **misclassified** points and those within the classification margin, aiming to **maximize** the **distance between data classes**.
Formula:
Purpose: it encourages the **model** to be **confident**, driving the SVM to find a **hyperplane** that **maximizes the gap between classes**.

Also:

- it less sensitive to outliers compared to functions like MSE, as it stop penalizing correct "confident" prediction